JSON

Status: not covered by Software Lab-1 practicals **Where it is used:** `backend/analytics.py` (`json_check`), `tests/test_analytics.py` (`assert_json_safe`, four `test_*` functions), `tests/test_app.py` (`data`, `test_every_response_is_strict_json`) **Closest lab concept:** Practical 4, file handling — a text file has no rules about what goes inside it; JSON is the same idea with rules attached

---

1. What it is

JSON is a text format for structured data. "Structured" is the whole point: the text is not free-form, it has a fixed grammar, and a parser reading that grammar always recovers the same types you put in.

Concretely, a Lab 4 text file written with `open()`/`write()` has no rules at all. Anything goes, in any order, and the reader has to know by convention what column 3 meant:

```
P-0001 Aarav 41
P-0002 Divya 34
```

Move a value and the file is silently wrong. JSON fixes that by demanding three things of you:

1. **Names for every value.** Keys are mandatory in an object, so a value can never lose its label.
2. **A declared type per value.** A number looks like a number, a string like a string. `7` and `"7"` are different things.
3. **An escaping rule.** Special characters inside a string must be escaped, so a string cannot accidentally run into the structure around it.

The mapping is small and fixed:

Python	JSON	Written as
`dict`	object	`{"k": v}`
`list`, `tuple`	array	`[v, v]`
`str`	string	`"text"`
`int`, `float`	number	`768`, `120.89`
`bool`	boolean	`true` / `false`
`None`	null	`null`

Three common values have **no** JSON spelling at all: `numpy.int64`, `NaN` and `NaT`. Section 4 is about those, and about the `to_native()` function the project wrote to absorb all three before a serialiser ever sees them.

Note that JSON is written in lowercase `true`, `false`, `null`. Python's `True`, `False`, `None` are what you *type*; `true`, `false`, `null` is what *arrives on the wire* and is what `json.loads` gives back as `True`, `False`, `None`.

---

2. Why the project needed it

Two reasons, one structural and one that forced a design decision.

The structural reason. `backend/analytics.py` runs pandas aggregations and hands the results straight to Flask. Flask's `jsonify` is a serialiser: it takes a Python object and turns it into a JSON response body. That conversion is not optional, so every value `analytics.py` returns has to be something the converter accepts.

The real reason. pandas does not hand back plain Python objects. It hands back values from its own numeric library, and it represents a missing number as `NaN` ("not a number", a floating point sentinel, not

the absence of anything). Neither is JSON. `analytics.py` states the rule at the top of the file:

```
Everything returned from here must survive json.dumps. pandas produces
numpy types and NaN, neither of which is valid strict JSON, so every
public function ends with to_native() and the test suite asserts
json.dumps(..., allow_nan=False) succeeds.
```

That is why `to_native()` exists, and why there is a whole test file section about serialisation instead of just `assert result == expected`.

---

3. The three pieces actually used

The project imports `json` in exactly three files, and in `backend/` it is imported in exactly one. Worth knowing up front: **the backend never builds a JSON string by hand.** `jsonify` does that. The only `json.dumps` call in `backend/` lives inside a *checking* function.

3.1 `json.dumps` — Python object into a JSON string

```
def json_check(payload):
    """Return (ok, message). Used by tests to prove a payload is safe."""
    try:
        json.dumps(payload, allow_nan=False)
        return True, "serialises as strict JSON"
    except (TypeError, ValueError) as e:
        return False, str(e)
```

`dumps` means "dump to a string". Note that it raises rather than returning anything useful on failure, so it is wrapped in `try`. The two exception types are exactly the two failure modes from section 4: `TypeError` for a type it does not know, `ValueError` for a float it knows but refuses to write.

Measured, on this project's Python 3.14.2 / numpy 2.4.1 / pandas 3.0.5:

```
json_check({"a": 1})            -> (True, 'serialises as strict JSON')
json_check({"a": np.int64(1)})  -> (False, 'Object of type int64 is not JSON serializable')
json_check({"a": float("nan")}) -> (False, 'Out of range float values are not JSON compliant: nan')
```

3.2 `json.loads` — the reverse, and the reader side

`loads` means "load from a string". It is the counterpart, and it appears in the tests, where the browser's side of the conversation is simulated:

```
def data(response):
    return json.loads(response.data)
```

`response.data` is raw bytes. `json.loads` turns them back into a dict so the test can assert on `body["datasets"]` and friends, exactly as the frontend's JavaScript would after `JSON.parse`.

3.3 `json.dumps(obj, indent=2)` — for reading, not for sending

`indent=2` pretty-prints with two-space indentation. It changes nothing semantically; the parsed result is identical. It is for humans at a terminal.

`indent` appears nowhere in this repository, and that is a deliberate choice: whitespace inside a response body is wasted bytes, and `jsonify` already ships compact output. It is worth knowing because it is how you actually look at a payload while debugging:

```
import sys
sys.path.insert(0, "backend")
import analytics, json

print(json.dumps(analytics.profile("pima_diabetes.csv"), indent=2)[:180])
```

```
{
  "name": "pima_diabetes.csv",
  "rows": 768,
  "cols": 9,
  "target": "Outcome",
  "numeric_columns": [
    "Pregnancies",
    "Glucose",
    "BloodPressure",
    "SkinThickness
```

That is the real first 180 characters of the real profile payload, cut mid-token by the `[:180]`, unmodified.

---

4. The three serialisation traps

4.1 `numpy.int64` is rejected, `numpy.float64` is accepted

pandas is built on numpy, and integer columns come out as `numpy.int64`, which is a *different type* from Python's `int`. `json.dumps` looks types up in a table, `numpy.int64` is not in it, so it raises.

```
    ok, msg = analytics.json_check({"a": np.int64(1)})
    assert ok is False, "int64 should have been rejected"
    assert "int64" in msg, msg
```

That is verbatim from `test_json_check_detects_bad_payloads` in `tests/test_analytics.py`.

`numpy.float64` is the interesting asymmetry, and it is not obvious from the outside. `float64` *subclasses* Python's `float`; `int64` does not subclass `int`. Verified directly:

```
issubclass(np.float64, float)  ->  True
issubclass(np.int64, int)      ->  False

json.dumps({"a": np.float64(1.5)})  ->  '{"a": 1.5}'
json.dumps({"a": np.int64(1)})      ->  TypeError: Object of type int64 is not JSON serializable
```

So a `float64` sails through and an `int64` blows up, in the same dict, from the same library. Relying on "pandas floats are fine" is a trap: a count, a row index or a length is an integer, so it is the value most likely to be `int64`. `analytics.py` therefore casts at the source rather than hoping:

```
def value_counts(df, column, limit=10):
    """Return the most common values in a column."""
    if column not in df.columns:
        return {}

    counts = df[column].value_counts(dropna=True)
    trimmed = counts.head(limit)

    return to_native({str(k): int(v) for k, v in trimmed.items()})
```

`int(v)` removes the numpy type outright. The test that locks this in is `test_profile_rows_are_plain_ints`, which asserts on the *type*, not the value:

```
def test_profile_rows_are_plain_ints():
    """shape values must be int, not numpy int."""
    p = analytics.profile("pima_diabetes.csv")

    assert isinstance(p["rows"], int), type(p["rows"])
    assert isinstance(p["cols"], int), type(p["cols"])
```

An honest footnote: with pandas 3.0.5, `to_dict()` already boxes scalars to native Python, so most of these casts are now double insurance. The guard is still worth its cost — the cost of a version that boxes is one cast, the cost of a version that does not is a 500 on every analytics route — and

`test_json_check_detects_bad_payloads` keeps the guard itself under test regardless of what pandas happens to return this month.

4.2 `NaN` is worse than `int64`, because it fails silently

`int64` at least raises. `NaN` does not. By default `json.dumps` writes it out as a bare `NaN` token:

```
json.dumps({"x": float("nan")})   ->   '{"x": NaN}'
```

`NaN` is not valid JSON. The JSON grammar has no such token. So the function that claims to serialise JSON produces something no conforming parser will accept, and it does so without raising, which means a test that only checks "did `dumps` succeed" will pass on broken output.

Worse, Python's own reader accepts its own non-standard output:

```
json.loads('{"x": NaN}')   ->   {'x': nan}
```

Round-tripping through `dumps` and `loads` in the same process hides the bug completely. This is why the frontend is where it would have surfaced: a real `JSON.parse` in a browser rejects `NaN`, and the page renders nothing.

The project hits `NaN` at the source, too. `pandas-missing-values.md` covers why; the relevant point here is that `read_csv` with `na_values` deliberately *manufactures* `NaN` wherever the dataset printed `?` or `-9`.

4.3 `allow_nan=False` turns silence into a catchable error

The fix is one keyword argument:

```
json.dumps({"x": float("nan")}, allow_nan=False)
ValueError: Out of range float values are not JSON compliant: nan
```

Same for infinity:

```
json.dumps({"x": float("inf")})                  ->   '{"x": Infinity}'
json.dumps({"x": float("inf")}, allow_nan=False)  ->   ValueError: Out of range float values are not JSON
```

`allow_nan=False` does not sanitise anything. It refuses, loudly, so the problem surfaces where it can still be fixed. That makes `json.dumps(payload, allow_nan=False)` a genuine assertion — it is the difference between testing the payload and testing that `dumps` returned without throwing. `json_check` exists to package that into something a test can call with a readable failure message, and it is used as a gate in eight places in `tests/test_analytics.py`:

```
def test_profile_payloads_are_strict_json():
    """The phase gate: every profile must survive json.dumps."""
    for name in analytics.passing_datasets():
        ok, msg = analytics.json_check(analytics.profile(name))
        assert ok, f"{name}: {msg}"
    print("  ok  every profile serialises as strict JSON, allow_nan=False")
```

The helper that the other tests call:

```
def assert_json_safe(payload, label):
    """Fail loudly if a payload is not valid strict JSON.

    allow_nan=False is what makes this a real check. By default json.dumps
    emits a bare NaN token, which is not valid JSON and breaks strict
    parsers on the frontend.
    """
    ok, message = analytics.json_check(payload)
    assert ok, f"{label} is not strict JSON: {message}"
```

4.4 `to_native()` — fixing it before it becomes a problem

`allow_nan=False` tells you a payload is broken. `to_native()` stops it being broken. It is the boundary function: pandas values go in, plain Python comes out.

```
def to_native(value):
    """Convert pandas and numpy values into plain Python.

    numpy int64 is not JSON serialisable. numpy float64 happens to be,
    because it subclasses float, but it is converted anyway so the output
    type is predictable. NaN and NaT become None, which is what JSON null
    means, rather than the bare NaN token json.dumps emits by default.

    Takes Any on purpose: it is a boundary function whose whole job is to
    accept whatever pandas produced and return something safe.
    """
    if value is None:
        return None

    if isinstance(value, (bool,)):
        return bool(value)

    if isinstance(value, int):
        return int(value)

    if isinstance(value, float):
        return None if pd.isna(value) else float(value)

    if isinstance(value, str):
        return None if value == "" else value

    if isinstance(value, dict):
        return {str(to_native(k)): to_native(v) for k, v in value.items()}

    if isinstance(value, (list, tuple)):
        return [to_native(v) for v in value]

    # Scalars only at this point. A Series or Index would make pd.isna
    # return an array, whose truth value is ambiguous.
    try:
        if pd.isna(value):
            return None
    except (TypeError, ValueError):
        pass

    if hasattr(value, "item"):
        return to_native(value.item())

    return value
```

Four decisions in there are worth defending:

`bool` is checked before `int`. `isinstance(True, int)` is `True` in Python, because `bool` subclasses `int`. Checking `int` first would turn `True` into `1` in the JSON, quietly losing the difference between `"passes": true` and `"passes": 1`. The test that pins this down asserts on type, not truth:

```
assert analytics.to_native(None) is None
    assert analytics.to_native(True) is True
    assert isinstance(analytics.to_native(True), bool)
```

It recurses. The `dict` and `list` branches call themselves, so a numpy integer six levels down in a payload is fixed the same way as one at the top. Keys are stringified too, because JSON object keys must be strings and a pandas `describe()` frame indexes columns by label.

`NaN` becomes `None`, and the key is kept. This is the point the module docstring opens with. The alternative — dropping the key entirely — makes `"chol": null` and no `chol` key indistinguishable, which for a clinical reading is the difference between *we measured zero* and *we never measured*. `null` is the

honest answer: the field exists, the value is unknown.

`.item()` is the fallback. Every numpy scalar has a `.item()` method that returns the equivalent Python scalar, so it converts anything the explicit branches missed. And `to_native` returns its own input unchanged as a last resort rather than raising, because a boundary function's job is to be total.

The test that pins the NaN half down, including the proof that the conversion matters:

```
def test_to_native_converts_nan_to_none():
    """NaN becomes None so JSON says null rather than NaN."""
    import numpy as np

    assert analytics.to_native(float("nan")) is None
    assert analytics.to_native(np.float64("nan")) is None
    assert analytics.to_native({"a": float("nan")}) == {"a": None}
    assert analytics.to_native([float("nan"), 1.0]) == [None, 1.0]

    # Proof the conversion matters: raw NaN is rejected by strict JSON.
    try:
        json.dumps({"x": float("nan")}, allow_nan=False)
        raise AssertionError("expected raw NaN to be rejected by allow_nan=False")
    except ValueError:
        pass

    assert_json_safe(analytics.to_native({"x": float("nan")}), "NaN payload")
    print("  ok  NaN becomes None and never reaches json.dumps")
```

---

5. Why this matters here: `jsonify` inherits every one of these

`backend/app.py` does its serialisation through Flask, not through `json`:

```
return jsonify(analytics.profile(valid))
```

```
return jsonify(analytics.risk_bands(valid, column))
```

`jsonify` calls `json.dumps` internally, so it has exactly the same two weaknesses — and it is where they stop being a test failure and become a broken API. Measured against a real Flask 3.0.0 app:

What the route returns	Result
`jsonify({"x": None})`	`200`, body `{"x":null}`
`jsonify({"x": float("nan")})`	`200`, body `{"x":NaN}` — **invalid JSON, no error raised**
`jsonify({"x": np.int64(1)})`	`500 Internal Server Error`

The `int64` row is a visible, obvious outage. The `NaN` row is the dangerous one: the route reports success, the status code says 200, and the analytics endpoints would each ship a body the browser cannot parse — with nothing in the logs to say why.

That is why `to_native()` is not defensive decoration. It is the only thing standing between a pandas `NaN` and a dashboard that renders empty.

---

6. Text format vs JSON: the same data, two consumers

`backend/triage.py` also returns text, and that text is **not** JSON:

```
def render_report(patient, results, conditions=None):
    """Render the consultation report in the Lab 5 banner format.

    patient is a dict with at least patient_id, name and optionally age.
    The disclaimer is always the final line, per spec section 5.5.
    """
    line = "*" * 44
```

```
    out = []
    out.append(line)
    out.append(" MERIDIAN CONSULTATION")
    out.append(line)
```

Real output, 908 characters, a plain Python `str`:

```
****************************************
 MERIDIAN CONSULTATION
****************************************
Patient       : P-0001  (Aarav)  age 41

MOST LIKELY

  1. Influenza               66%   match 3/7   MODERATE
     evidence: moderate   medication: Oseltamivir 75mg twice daily for 5 days
  2. COVID-19                66%   match 3/7   MODERATE
     evidence: moderate   medication: Paracetamol 500mg every 6 hours
  3. Malaria                  0%   match 2/8   SEVERE
     evidence: weak   medication: Artemisinin-based combination as directed by clinician
  4. Asthma                   1%   match 2/5   MODERATE
     evidence: weak   medication: Salbutamol inhaler 2 puffs as needed
     (listed below Malaria on score because severity outweighs a 0.01 difference)

****************************************
NOT A DIAGNOSIS. TRIAGE GUIDANCE ONLY.
****************************************
```

The `*` banners, the column alignment and the `{r['name']:<22}` padding all exist to be *looked at*. A JSON object could not carry that layout, because JSON has no concept of a fixed-width column. In the other direction, this string cannot be queried: you cannot ask "what is the match count for rank 2" without re-parsing the text, whereas `results[1]["matched_count"]` is just a dict lookup.

The interesting part is that both travel over the same HTTP response, because `app.py` puts the report *inside* a JSON object as a string value:

```
return jsonify({
    "count": len(results),
    "results": results,
    "warnings": warnings,
    "patient": patient,
    "report": triage.render_report(patient or {"name": "anonymous"}, results,
                                   conditions),
    "disclaimer": triage.DISCLAIMER,
})
```

So the report is text, but it gets escaped on the way out. The report holds 19 newlines, and each one is replaced by the two characters `\` and `n`:

```
"****************************************\n MERIDIAN CONSULTATION\n********************************
```

The `...` is an elision; everything before it is verbatim, including the opening `"` that marks the whole thing as a single JSON string value. That is JSON's escaping rule from section 1 doing its job. One response, two representations, one consumer each: `results` and `patient` are for the browser's JavaScript to index into, and `report` is for a human to read in a `<pre>` block. Neither format can be removed without breaking something.

---

7. Questions this should answer

- What does `json.dumps` do, and what does `json.loads` do?
- Why can `json.dumps({"a": np.float64(1.5)})` succeed while the same call with `np.int64(1)` raises

`TypeError`?

- What is the default output of `json.dumps({"x": float("nan")})`, and why is it a problem even though nothing raised?
- What does `allow_nan=False` change about a call to `json.dumps`?
- Why does `to_native()` check `bool` before `int`?
- Why does a missing measurement become `null` instead of having its key dropped?
- `test_every_response_is_strict_json` in `tests/test_app.py` parses each response with a `parse_constant` hook that raises. Why is a hook needed, given that `json.loads` happily reads a bare `NaN`?